

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Báo cáo Thực tập chuyên ngành

Đánh giá phôi đại VA mức video từ nội soi mũi họng với phân
đoạn hai vùng

Giảng viên hướng dẫn:

Giảng viên đồng hướng dẫn:

Lớp:

Sinh viên:

Mã sinh viên:

Dr. Lê Đức Trọng

Dr. Đỗ Thành Công

QH-2032-I/CQ-I-CS3

Phạm Mai Anh

23021469

Hà Nội, 2026

MỤC LỤC

Lời cảm ơn	3
1 Giới thiệu chung	4
1.1 Giới thiệu công việc	4
1.2 Giới thiệu qua bài toán	4
2 Yêu cầu bài toán	5
2.1 Bài toán lớn: đánh giá phi đại VA ở mức video	5
2.2 Phần bài toán sinh viên giải quyết	5
2.3 Tiêu chí đánh giá	6
3 Tóm tắt lý thuyết, giải pháp, thuật toán	6
3.1 Các lý thuyết, giải pháp, thuật toán liên quan	6
3.2 Cách giải quyết của sinh viên	7
3.2.1 Bộ nhớ không gian Kalman một khe	7
3.2.2 Baseline EMA gain hằng	9
3.2.3 Huấn luyện bộ cập nhật	9
3.2.4 Hạ tầng dữ liệu và đánh giá	10
3.3 Liên hệ và so sánh với các cách đã có	11
4 Mô tả phần mềm cài đặt	12
4.1 Cấu trúc mã nguồn	12
4.2 Cài đặt	13
4.3 Quy trình chạy	13
5 Kết quả đạt được, hướng phát triển	13
5.1 Kết quả đạt được	13
5.1.1 Thiết lập thực nghiệm	13
5.1.2 Kết quả trên C6	14
5.1.3 Gain Kalman so với gain hằng	15
5.1.4 Bộ nhớ và tốc độ	15

5.1.5	Đánh giá hai vùng trên REFUGE	16
5.2	Kỹ năng và kiến thức thu thập được	17
5.3	Hướng phát triển tiếp theo	17
Tài liệu tham khảo		18

Lời cảm ơn

Tôi xin chân thành cảm ơn các giảng viên của Trường Đại học Công nghệ, đặc biệt là Dr. Lê Đức Trọng và Dr. Đỗ Thành Công, đã hướng dẫn và góp ý cho quá trình thực hiện đề tài. Tôi cũng cảm ơn các bạn và những người đã hỗ trợ trao đổi chuyên môn trong thời gian thực tập.

1 Giới thiệu chung

1.1 Giới thiệu công việc

Phì đại VA (adenoid hypertrophy) là một nguyên nhân thường gặp gây tắc nghẽn đường thở trên ở trẻ em. Trong thực hành, bác sĩ đánh giá mức độ phì đại bằng nội soi mũi họng: ống soi đi qua hốc mũi, và mức độ VA che lấp lòng mũi họng được ước lượng bằng mắt. Cách đánh giá này phụ thuộc người đọc và thường chỉ dựa trên một vài khung hình được chọn. Đề tài thực tập hướng tới việc tự động hóa phép đo này ở *mức video*: phân đoạn vùng VA và vùng đường thở mũi họng trên từng khung hình, tính tỷ số tắc nghẽn trên các khung hợp lệ, rồi tổng hợp thành một đánh giá cho cả video.

Trong đợt thực tập, công việc của tôi tập trung vào phần kỹ thuật cốt lõi của chuỗi xử lý đó: (i) hạ tầng dữ liệu video dùng chung cho huấn luyện, suy luận và đánh giá, kể cả bộ chuyển đổi mã hóa mức xám kiểu dữ liệu VA của Cai và cộng sự [3]; (ii) thiết kế và cài đặt bộ nhớ không gian Kalman một khe thay cho ngân hàng bộ nhớ FIFO bảy khung của MedSAM2 [1], cùng baseline EMA để đối chứng; (iii) bộ đánh giá theo thời gian và bộ đánh giá hai vùng có sai số tỷ số; (iv) thiết kế giao thức thực nghiệm, chạy thí nghiệm, phân tích lỗi và ghi nhận trung thực cả các kết quả âm.

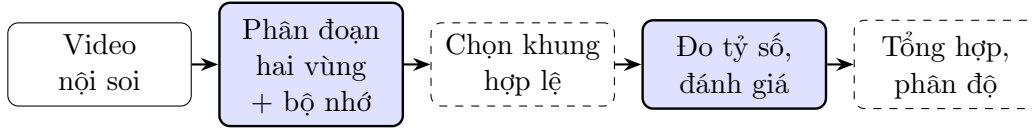
Ở thời điểm báo cáo, dự án chưa có bộ dữ liệu video VA đã gán nhãn, cũng chưa có giao thức lâm sàng xác định công thức tắc nghẽn và ngưỡng phân độ. Vì vậy mọi thực nghiệm dùng dữ liệu thay thế: PolypGen [4] cho bài toán lan truyền phân đoạn theo thời gian, REFUGE [5] cho đường xử lý hai vùng và sai số tỷ số. Báo cáo không tuyên bố kết quả lâm sàng trên VA.

1.2 Giới thiệu qua bài toán

Chuỗi xử lý mục tiêu của dự án AdeSEG được minh họa ở Hình 1:

video nội soi $\rightarrow$ phân đoạn VA và đường thở $\rightarrow$ chọn khung hình hợp lệ
 $\rightarrow$ đo tắc nghẽn theo khung hình $\rightarrow$ tổng hợp theo video $\rightarrow$ phân độ.

Bài toán khó ở hai điểm. Thứ nhất, phân đoạn phải *ổn định theo thời gian*: video nội soi có chuyển động camera mạnh, chiếu sáng thay đổi, khói hoặc dịch che khuất, và đối tượng có thể ra khỏi khung hình rồi xuất hiện lại. Một mô hình phân đoạn video có bộ nhớ như MedSAM2 có thể “bám nhầm” đối tượng và tích lũy lỗi qua các khung. Thứ hai, chỉ số lâm sàng là một *tỷ số diện tích* giữa hai vùng, nên Dice tốt của từng vùng chưa chắc cho tỷ số chính xác. Đề tài giải quyết điểm thứ nhất bằng một kiến trúc bộ nhớ mới, và điểm thứ hai bằng một bộ đánh giá đo trực tiếp sai số tỷ số.



Hình 1: Chuỗi xử lý mục tiêu. Ô tô đậm: phần sinh viên đã cài đặt và đánh giá; ô nét đứt: phần có khung mã nguồn nhưng cần giao thức lâm sàng và dữ liệu VA để hoàn thiện.

2 Yêu cầu bài toán

2.1 Bài toán lớn: đánh giá phì đại VA ở mức video

Cho một video nội soi gồm các khung $I_1, \dots, I_T$, hệ thống cần dự đoán cho mỗi khung hai mặt nạ nhị phân: A_t cho VA và B_t cho đường thở mũi họng không bị che. Một khung chỉ được dùng để đo khi hai mặt nạ cùng kích thước, có diện tích đủ lớn và tỷ số hữu hạn. Kho mã nguồn hỗ trợ hai dạng tỷ số diện tích:

$$r_t = \frac{|A_t|}{|A_t| + |B_t|} \quad \text{hoặc} \quad r_t = \frac{|A_t|}{|B_t|}.$$

Dạng thứ nhất có cùng ý nghĩa với tỷ số A/N mà Cai và cộng sự [3] dùng để phân độ: nhỏ ($< 50\%$), trung bình ($50-75\%$), lớn ($> 75\%$). Từ các khung hợp lệ, tỷ số của video được tổng hợp bằng trung vị hoặc trung bình, rồi ánh xạ sang mức độ phì đại bằng một dãy ngưỡng tăng chặt. Công thức tỷ số và ngưỡng là lựa chọn của giao thức lâm sàng, nên mã nguồn buộc người dùng truyền chúng một cách tường minh thay vì cài sẵn một giá trị mặc định.

Đầu vào. Video đã sắp theo thứ tự khung. Mỗi khung là một dòng manifest JSONL với các trường `split`, `video_id`, `frame_index`, `image`, `mask`. Mặt nạ là ảnh chỉ số: 0 nền, 1 đường thở, 2 VA (khớp với mã hóa mức xám 0/128/255 của Cai và cộng sự sau khi chuyển đổi). Việc chia tập phải theo video hoặc bệnh nhân, không theo khung, để tránh rò rỉ dữ liệu.

Đầu ra. Ở mức khung: mặt nạ từng vùng, diện tích, tỷ số, kiểu tỷ số và cờ `valid_frame`. Ở mức video: tỷ số tổng hợp và, khi có ngưỡng, nhãn phân độ.

2.2 Phần bài toán sinh viên giải quyết

Đề tài được thực hiện cá nhân dưới sự hướng dẫn của hai giảng viên; mọi phần cài đặt, thực nghiệm và phân tích trong báo cáo là công việc của sinh

viên. Trong bài toán lớn ở trên, tôi giải quyết hai phần cụ thể:

(a) Bộ nhớ theo thời gian cho phân đoạn video. MedSAM2 lan truyền mặt nạ từ khung có prompt sang các khung sau nhờ một ngăn hàng bộ nhớ chứa tối đa bảy khung gần nhất (FIFO). Khi một khung bị phân đoạn sai, bộ nhớ sai đó được ghi vào ngăn hàng và ảnh hưởng tới các khung sau; khi đối tượng biến mất, mô hình vẫn có xu hướng dự đoán đối tượng. Yêu cầu đặt ra là thay ngăn hàng FIFO bằng một kiến trúc bộ nhớ có kích thước cố định, có cơ chế quyết định mức độ ghi mỗi khung vào bộ nhớ, và chứng minh bằng thực nghiệm rằng thay đổi này giúp giảm mất theo dõi mà không làm tăng dương tính giả. Ràng buộc: giữ nguyên trọng số MedSAM2 (đông lạnh), chỉ thay cơ chế bộ nhớ, để so sánh công bằng.

(b) Đánh giá hai vùng và sai số đo. Xây dựng đường xử lý hai nhân từ mặt nạ mã hóa mức xám đến tỷ số, và một bộ đánh giá báo cáo đồng thời Dice/IoU từng vùng và sai số tỷ số. Mục đích là kiểm tra trực tiếp liệu Dice tốt có thể đi kèm sai số tỷ số lớn hay không, trước khi đề xuất bất kỳ hàm mất mát “nhận biết phép đo” nào.

2.3 Tiêu chí đánh giá

Phân đoạn được đo bằng Dice và IoU theo từng chuỗi rồi lấy trung bình, cùng Dice riêng trên các khung có đối tượng. Hiện diện được đo bằng tỷ lệ dương tính giả trên khung rỗng (FP khung rỗng). Tích lũy lỗi được đo bằng tỷ lệ khung có đối tượng bị “mất” ($\text{Dice} < 0.1$), độ dài trung bình một đoạn mất, tỷ lệ đoạn được khôi phục và tỷ lệ mất tới cuối video. Độ bền được đo bằng Dice ở 1–10 khung sau khi đối tượng tái xuất hiện và ở khung chuyển động cao. Đo hai vùng dùng MAE, RMSE và tương quan Pearson/Spearman giữa tỷ số dự đoán và tỷ số thật, cùng độ chính xác phân độ và hệ số kappa khi có ngưỡng. So sánh cặp theo chuỗi dùng kiểm định Wilcoxon kèm khoảng tin cậy bootstrap theo chuỗi; trung bình theo chuỗi và trung bình gộp theo khung luôn được ghi rõ.

3 Tóm tắt lý thuyết, giải pháp, thuật toán

3.1 Các lý thuyết, giải pháp, thuật toán liên quan

SAM2 và MedSAM2. SAM2 [2] phân đoạn video bằng một bộ mã hóa ảnh, một khối *memory attention* cho đặc trưng khung hiện tại chú ý tới các bộ nhớ đã lưu, một bộ giải mã mặt nạ và một bộ mã hóa bộ nhớ. Sau mỗi

khung, bộ mã hóa bộ nhớ nén đặc trưng ảnh và mặt nạ dự đoán thành một bản đồ $64 \times 32 \times 32$; bản đồ này được đẩy vào ngân hàng bộ nhớ gồm khung prompt cộng tối đa sáu khung gần nhất, cùng các *object pointer*. Bộ giải mã còn xuất một *object score* cho biết đối tượng có mặt hay không. MedSAM2 [1] tinh chỉnh SAM2 trên dữ liệu ảnh và video y tế, giữ nguyên kiến trúc bộ nhớ này. Điểm yếu đã biết: mọi khung đều được ghi vào ngân hàng với cùng trọng số, kể cả khung bị phân đoạn sai.

Cải tiến bộ nhớ của SAM2. SAM2Long [6] giữ nhiều nhánh giả thuyết bộ nhớ và chọn nhánh tốt theo điểm tích lũy. DAM4SAM [7] tách bộ nhớ thành phần gần đây và phần chống nhiễu (distractor-aware), chỉ ghi khung khi đủ tin cậy. EMA-SAM [8] dùng trung bình trượt có trọng số theo độ tin cậy cho bộ nhớ của SAM2 trong ảnh y tế. SAMURAI [9] dùng bộ lọc Kalman, nhưng trên *chuyển động hộp bao* để chọn mặt nạ ứng viên và lọc bộ nhớ, không phải trên chính đặc trưng bộ nhớ.

Bộ nhớ lặp kích thước cố định. Trong phân đoạn đối tượng video, RDE-VOS [10] thay ngân hàng bộ nhớ ngày càng lớn bằng một embedding động được cập nhật lặp; LiVOS [11] dùng ma trận trạng thái tuyến tính có cống.

Bộ lọc Kalman và Kalman học được. Bộ lọc Kalman [12] ước lượng trạng thái ẩn từ các quan sát nhiễu bằng hai bước dự đoán và cập nhật; gain K cân bằng giữa trạng thái cũ và quan sát mới theo phương sai của chúng. Recurrent Kalman Networks (RKN) [13] đưa cập nhật Kalman với hiệp phương sai đường chéo vào không gian đặc trưng học được; KalmanNet [14] học trực tiếp gain bằng mạng nơ-ron. Bộ nhớ trong đề tài dùng đúng phương trình cập nhật của RKN, đặt lên bản đồ bộ nhớ không gian của MedSAM2.

3.2 Cách giải quyết của sinh viên

3.2.1 Bộ nhớ không gian Kalman một khe

Ý tưởng: thay ngân hàng FIFO bằng (i) bộ nhớ cố định của khung prompt (anchor) và (ii) *một* trạng thái không gian gồm trung bình $S \in \mathbb{R}^{64 \times 32 \times 32}$ và phương sai theo pixel $P \in \mathbb{R}^{32 \times 32}$ (hiệp phương sai đường chéo, dùng chung cho 64 kênh tại mỗi pixel). Mô hình chuyển trạng thái và mô hình quan sát

là đơn vị. Tại khung t :

dự đoán: $P_t^- = P_{t-1} + q$,

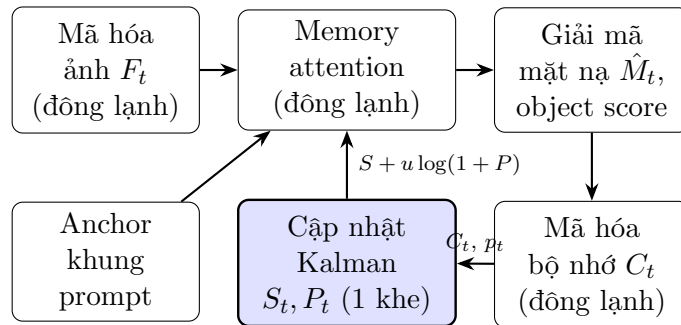
đọc: $[\text{anchor} \mid S_{t-1} + u \log(1 + P_t^-)]$ cùng object pointer,

quan sát: $C_t =$ bộ mã hóa bộ nhớ MedSAM2($F_t, \hat{M}_t$),

nhieu: $R_t = (r + a(1 - p_t)) c(D_t - \overline{D_t})$,

cập nhật: $K_t = \frac{P_t^-}{P_t^- + R_t}$, $S_t = S_{t-1} + K_t (C_t - S_{t-1})$, $P_t = (1 - K_t) P_t^-$.

Ở đây F_t là đặc trưng ảnh, $\hat{M}_t$ là mặt nạ dự đoán, p_t là xác suất hiện diện suy ra từ object score, u là một embedding bất định để memory attention “biết” vùng nào của trạng thái kém tin cậy, và D_t là một bản đồ hiệu chỉnh nhieu theo không gian có thể huấn luyện, $c(x) = 10^{\tanh x}$ (bị chặn bởi tanh nên chỉ nhận R trong khoảng $[0.1, 10]$). Khi $D_t = 0$, mô-đun chưa huấn luyện chính là bộ lọc đặt tay với $q = 1$, $r = 0.1$, $a = 10$, $P_0 = 1$: khung có đối tượng ghi gần như toàn bộ quan sát vào trạng thái, khung rỗng chỉ ghi một phần nhỏ. **Trong mọi lần chạy được báo cáo, các phần theo pixel đều chưa được huấn luyện:** lớp cuối của D_t và u bằng 0, nên R_t là một số vô hướng cho mỗi khung, P đồng nhất trong không gian và không bao giờ được đọc. Như vậy, phương pháp *thực sự được chạy* là một bộ lọc Kalman vô hướng trên bộ nhớ một khe, với nhieu quan sát chuyển theo xác suất hiện diện; vai trò “cho memory attention biết vùng nào kém tin cậy” của u chỉ là khả năng của mã nguồn, chưa được kiểm chứng. Anchor dùng mã thời gian của khung điều kiện trong MedSAM2, còn trạng thái dùng mã thời gian của khung gần nhất, nên memory attention đồng lạnh nhận đầu vào cùng định dạng như khi đọc ngân hàng gốc. Hình 2 so sánh hai kiến trúc.



Hình 2: Luồng dữ liệu của bộ nhớ một khe: MedSAM2 đọc anchor của khung prompt cộng một trạng thái lặp thay vì ngân hàng FIFO bẫy khung; chỉ khối tô đậm là mới, toàn bộ MedSAM2 giữ đông lạnh.

Tùy chọn bỏ qua khung vắng mặt. Với cờ `-skip_absent`, khung mà MedSAM2 gọi là rỗng được xem là *thiếu quan sát*: trạng thái không gian chỉ chạy bước dự đoán ($K = 0$), nên phương sai tăng dần trong đoạn vắng mặt và gain tự động lớn khi đối tượng xuất hiện lại. Đây là hành vi mà một gain hằng không có, và là giả thuyết chính về ưu thế của cập nhật Kalman. Khung vắng mặt cũng không đóng góp object pointer vào memory attention, nên đó là một quan sát thiếu hoàn toàn. Các kết quả chính không dùng chế độ này.

Phân tích trạng thái dừng. Với q hằng và R chỉ phụ thuộc vào trạng thái hiện diện, phương sai tiên nghiệm hội tụ về nghiệm của $M = MR/(M + R) + q$, tức

$$M^* = \frac{q + \sqrt{q^2 + 4qR}}{2}, \quad K^* = \frac{M^*}{M^* + R}.$$

Với $q = 1$: khi đối tượng có mặt ($R \approx 0.1$) thì $K^* \approx 0.92$; khi vắng mặt ($R \approx 10.1$) thì $K^* \approx 0.27$. Gain trung bình đo được trên tập phát triển là 0.835 và 0.266, khớp với phân tích. Như vậy, sau vài khung, bộ lọc chưa huấn luyện hoạt động gần như một trung bình trượt mũ (EMA) với hai gain hằng. Nhận xét này dẫn tới baseline đối chứng dưới đây.

3.2.2 Baseline EMA gain hằng

Để tách đóng góp của *cấu trúc một khe* khỏi đóng góp của *gain thích nghi*, tôi cài đặt `modeling/ema_memory.py`: cùng anchor, cùng trạng thái một khe và cùng đường đọc, nhưng gain là hằng số cho khung có mặt và khung vắng mặt, lấy đúng bằng gain trung bình của bộ lọc Kalman trên tập phát triển (0.835/0.266). Phương sai vẫn được cập nhật nhất quán $P \leftarrow (1 - k)P^-$ để đường đọc $S + u \log(1 + P)$ giống hệt. Vì gain của EMA lấy đúng từ gain trung bình của Kalman, việc hai phương pháp hòa nhau là điều *được kỳ vọng theo cách xây dựng*; chúng chỉ khác ở (i) gain chuyển tiếp ngay sau khi trạng thái hiện diện đổi, (ii) xác suất hiện diện mềm p_t (Kalman) so với ngưỡng cứng 0.5 (EMA), và (iii) chế độ `-skip_absent`. Phép so sánh với `-skip_absent` vì vậy mới là kiểm định đúng cho phần riêng của dạng Kalman.

3.2.3 Huấn luyện bộ cập nhật

Bộ cập nhật có thể huấn luyện với MedSAM2 đông lạnh, tái sử dụng `SAM2Train` và hàm mất mát `MultiStepMultiMasksAndIous` của MedSAM2 (focal 20, Dice 1 trên khung có đối tượng) cộng BCE cho object score tách thành phần hiện diện và vắng mặt; chỉ tham số của bộ cập nhật Kalman nhận gradient. `scripts/train_kalman.py` huấn luyện trên các đoạn video

tự nhiên từ một manifest; với PolypGen, một số video của tập phát triển được giữ lại để kiểm định (`-train_split dev -val_videos seq13 seq14 seq15`). Prompt ở khung đầu là hộp bao chặt từ nhân thật, giống hết khi suy luận (SAM2Train mặc định thêm nhiễu vào hộp, nên lớp huấn luyện thay bằng hộp không nhiễu). Một lần chạy thử hai bước trên PolypGen xác nhận đường huấn luyện hoạt động; các kết quả trong báo cáo dùng bộ lọc chưa huấn luyện (cờ `-untrained`).

3.2.4 Hạ tầng dữ liệu và đánh giá

Manifest và thứ tự khung. Tên tệp khung của PolypGen có hậu tố số với độ dài khác nhau, nên sắp theo bảng chữ cái là sai (khung 104 đứng trước khung 69). Phiên bản đầu của manifest mắc lỗi này và đưa 18/23 chuỗi vào sai thứ tự thời gian. Tôi phát hiện lỗi, sửa `build_polypgen_manifest.py` sắp theo số, chạy lại toàn bộ thí nghiệm, và thêm kiểm thử khẳng định thứ tự khung khi suy luận và khi đánh giá luôn trùng nhau.

Lỗi đồng bộ và kiểm chứng tương đương. Trên Apple MPS, việc MedSAM2 offload trạng thái với `non_blocking` đọc dữ liệu trước khi sao chép xong và làm hỏng các kết quả đầu; tôi sửa bằng cách giữ trạng thái trên thiết bị khi không dùng CUDA. Mã `native_pointers.py` được kiểm chứng trùng khớp từng bit với object pointer của MedSAM2 gốc, nên khác biệt giữa các phương pháp chỉ đến từ bộ nhớ không gian.

Tái lập kết quả và tính độc lập với máy. `scripts/analyze_runs.py` dựng lại mọi bảng PolypGen (trung bình theo chuỗi và gộp, đoạn mất, chuyển động cao, kiểm định Wilcoxon và khoảng bootstrap) từ các CSV theo khung đã lưu. Cờ `-untrained` tạo bộ lọc chưa huấn luyện mà không cần tải checkpoint; vì lớp cuối khởi tạo bằng 0, kết quả không phụ thuộc seed. MedSAM2 chỉ lấp lỗ nhỏ trong mặt nạ khi có phần mở rộng biên dịch `_C`; để kết quả không phụ thuộc máy, việc lấp lỗ được tắt tường minh. Chạy lại ba phương pháp trên C6 bằng mã hiện tại cho kết quả trùng từng khung với các lần chạy đã lưu.

Kiểm tra mặt nạ đầu vào. Mặt nạ được đọc kèm kiểm tra nhân: giá trị phải là 0 hoặc thuộc `-label_ids`, và mặt nạ JPEG mức xám nhiều lớp bị từ chối thay vì bị ngưỡng hóa thành nhị phân. Nếu không, một mặt nạ PNG 0/255 đọc với nhân 1 sẽ không có hộp prompt và cho mặt nạ rỗng mà không báo lỗi.

Mặt nạ hai vùng. Mặt nạ ba lớp mã hóa mức xám (0/128/255) nếu đọc như mặt nạ nhị phân sẽ gộp đường thở và VA thành một lớp. `build_mask_manifest.py` chuyển chúng sang PNG chỉ số bằng ánh xạ giá trị gần nhất (để nhiều nén JPEG không tạo lớp mới), ví dụ 0:0 128:1 255:2 cho dữ liệu kiểu Cai và cộng sự. `evaluation/two_region.py` tính Dice/IoU từng vùng và tính tỷ số dự đoán và tỷ số thật bằng *cùng* một `ratio_mode`, nên không thể vô tình so sánh hai tỷ số định nghĩa khác nhau. Cờ `-nested_labels` của `infer.py` prompt nhãn k như hợp của các nhãn $\geq k$ (đĩa thị = vành + cup; mũi họng = VA + đường thở) rồi lấy vùng ngoài trừ vùng trong.

3.3 Liên hệ và so sánh với các cách đã có

Bảng 1 đặt giải pháp cạnh các hướng liên quan. Về mặt mã nguồn, cập nhật Kalman đặt trực tiếp lên bản đồ bộ nhớ không gian của MedSAM2 với nhiều quan sát phụ thuộc trạng thái hiện diện, trong khi giữ nguyên toàn bộ trọng số đã huấn luyện. Như đã nêu, các phần theo pixel chưa từng được huấn luyện, nên phương pháp đã đánh giá là một bộ nhớ lặp một khe với gain vô hướng.

Bảng 1: So sánh định tính với các hướng liên quan.

Phương pháp	Cơ chế bộ nhớ	Kích thước	Quan hệ với đề tài
SAM2 / MedSAM2	FIFO: prompt + 6 khung gần nhất, ghi mọi khung	7 khung	Baseline chính
SAM2Long	Nhiều nhánh giả thuyết bộ nhớ	Nhiều bank	Chọn bộ nhớ, không gộp
DAM4SAM	Bộ nhớ gần đây + bộ nhớ chống nhiễu	Bank	Ghi có điều kiện
SAMURAI	Kalman trên hộp bao, chọn mặt nạ	Bank	Kalman trên chuyển động, không trên bộ nhớ
EMA-SAM	Trung bình trượt theo độ tin cậy	Cố định	Gần nhất với đề tài; chưa chạy
RDE-VOS, LiVOS	Trạng thái lặp / tuyến tính có cổng	Cố định	Cùng ý tưởng kích thước cố định
RKN, KalmanNet	Cập nhật Kalman trong không gian học được	–	Nguồn phương trình cập nhật
Đề tài	Anchor + 1 trạng thái, gain vô hướng theo hiện diện	0.754 MiB	–

So sánh thực nghiệm (mục 5) cho kết luận rõ ràng: lợi ích theo dõi so với ngân hàng FIFO đến từ *trạng thái lặp một khe*, và được một EMA gain

hằng đặt ngang bằng. Cách diễn đạt đúng là: *một bộ nhớ lập một khe; dạng Kalman không thêm gì đo được*. Điều này đặt giải pháp rất gần EMA-SAM, phương pháp gần nhất nhưng chưa được chạy để so sánh. Về bộ nhớ (đo lại bằng `infer.py` trên C6): MedSAM2 gốc giữ bộ nhớ của mọi khung, tăng khoảng 0.13 MiB mỗi khung (4.5–8.1 MiB trên C6), nhưng chỉ chú ý tới tối đa bảy bộ nhớ, tức $7 \times 64 \times 32 \times 32$ ở bf16 = 0.875 MiB; trạng thái một khe (anchor, mã vị trí, trung bình, phương sai ở float32) là 0.754 MiB cố định. Như vậy bộ nhớ một khe không nhỏ hơn một ngân hàng đã cắt tĩa; ưu điểm là kích thước cố định và chỉ hai bản đồ bộ nhớ được đọc thay vì bảy.

4 Mô tả phần mềm cài đặt

4.1 Cấu trúc mã nguồn

Mã nguồn viết bằng Python/PyTorch, tái sử dụng tối đa các lớp của MedSAM2 (đặt trong thư mục `MedSAM2/`) thay vì viết lại. Logic Kalman là một mixin dùng chung cho cả huấn luyện và bộ dự đoán video. Bảng 2 tóm tắt các mô-đun do sinh viên viết.

Bảng 2: Các mô-đun chính.

Mô-đun	Chức năng
<code>modeling/kalman_memory.py</code>	Bộ cập nhật Kalman, mixin, bộ dự đoán video
<code>modeling/ema_memory.py</code>	Baseline EMA gain hằng, cùng đường đọc
<code>modeling/native_pointers.py</code>	Object pointer, trùng khớp MedSAM2 gốc
<code>training/kalman_trainer.py</code>	<code>SAM2Train</code> cộng mixin Kalman
<code>datasets/video.py</code>	Đọc manifest, giải mã mặt nạ
<code>scripts/build_polypgen_manifest.py</code>	Manifest PolypGen, sắp khung theo số
<code>scripts/build_mask_manifest.py</code>	Mặt nạ mức xám sang PNG chỉ số
<code>scripts/infer.py, train_kalman.py</code>	Suy luận ba loại bộ nhớ; huấn luyện
<code>evaluation/temporal.py</code>	Chỉ số theo khung, hiện diện, mất theo dõi
<code>evaluation/two_region.py</code>	Dice/IoU từng vùng, sai số tỷ số, kappa
<code>scripts/analyze_runs.py</code>	Dựng lại mọi bảng PolypGen từ CSV
<code>adenoid/</code>	Đo tỷ số, chọn khung hợp lệ, phân độ
<code>tests/test_data_pipeline.py</code>	Kiểm thử thứ tự khung, giải mã mặt nạ

4.2 Cài đặt

Phần mềm chạy trên CUDA, Apple MPS hoặc CPU. Cài đặt gồm: tạo môi trường ảo Python và chạy `pip install -r requirements.txt`; đặt checkpoint `MedSAM2_latest.pt` vào `checkpoints/`; đặt dữ liệu dưới `data/` và dựng manifest bằng các script `build*_manifest.py`; kiểm tra bằng `python3 -m pytest tests`.

4.3 Quy trình chạy

Suy luận đọc manifest, chọn tập qua `-split`, nhãn qua `-label_ids` và kiểu bộ nhớ qua `-memory_backend`:

```
python3 scripts/infer.py \
  --sam2_cfg configs/sam2.1_hiera_t512.yaml \
  --sam2_checkpoint checkpoints/MedSAM2_latest.pt \
  --manifest data/polypgen_sequence.jsonl \
  --split test --label_ids 1 --memory_backend kalman \
  --untrained --output_dir runs/kalman
python3 evaluation/temporal.py \
  --output_mask_dir runs/kalman/masks --sequences seq16 ... seq23
```

Thêm `-skip_absent` để bật chế độ bỏ qua khung vắng mặt; dùng `-fixed_gain 0.835 0.266` để chạy baseline EMA; `-memory_backend native` cho MedSAM2 gốc. Với dữ liệu hai vùng, mỗi nhãn được lan truyền trong một lượt riêng rồi gộp theo logit lớn nhất, hoặc theo thứ tự lồng nhau với `-nested_labels`; `two_region.py` chấm điểm với `-region_a`, `-region_b` và `-ratio_mode`. Mỗi lần suy luận ghi mặt nạ, bản ghi prompt, bộ nhớ giữ lại và tốc độ khung hình, và một CSV chẩn đoán theo khung cho từng nhãn (object score, xác suất hiện diện, gain trung bình, mức thay đổi đặc trưng ảnh), đủ để phân biệt lỗi phân đoạn, lỗi hiện diện và lỗi cập nhật bộ nhớ.

5 Kết quả đạt được, hướng phát triển

5.1 Kết quả đạt được

5.1.1 Thiết lập thực nghiệm

Thí nghiệm chính dùng tập C6 của PolypGen (seq16–seq23, trung tâm C6): 8 chuỗi nội soi đại tràng, 372 khung lan truyền, trong đó 56 khung không có polyp. Tập phát triển seq1–seq15 (trung tâm C1–C5) chỉ dùng để chọn thiết kế và đo gain của EMA (0.835/0.266). MedSAM2 giữ đông lạnh; prompt là hộp bao chặt của mặt nạ thật ở khung có polyp đầu tiên, nên kết

quả phản ánh chất lượng *lan truyền* sau prompt, không phải phát hiện tự động. Ba phương pháp được so sánh: MedSAM2 gốc (ngân hàng bảy khung), EMA gain hằng (một khe) và bộ nhớ Kalman chưa huấn luyện (một khe). Mọi lần chạy dùng một seed, không lắp lỗ mắt nạ, trên Apple MPS. C6 đã được xem trong quá trình phát triển, nên đây là đánh giá trên tập ngoài trung tâm phát triển chứ chưa phải một tập kiểm tra hoàn toàn chưa thấy.

Bảng 3: Các chuỗi C6: số khung lan truyền, số khung rỗng và Dice theo chuỗi.

Chuỗi	Khung	Rỗng	Dice			Dice khung polyp		
			Native	EMA	Kalman	Native	EMA	Kalman
seq16	33	1	0.941	0.940	0.942	0.970	0.970	0.971
seq17	46	3	0.812	0.807	0.814	0.869	0.863	0.871
seq18	62	7	0.594	0.640	0.655	0.560	0.649	0.647
seq19	55	1	0.830	0.846	0.847	0.827	0.843	0.844
seq20	35	5	0.143	0.218	0.143	0.000	0.088	0.000
seq21	47	23	0.685	0.426	0.448	0.757	0.751	0.751
seq22	45	2	0.678	0.712	0.713	0.710	0.746	0.746
seq23	49	14	0.253	0.238	0.258	0.097	0.076	0.076

5.1.2 Kết quả trên C6

Bảng 4 tổng hợp trung bình theo chuỗi; “mất” là tỷ lệ khung polyp có Dice < 0.1, “tái xuất hiện” là Dice ở 1–10 khung sau một đoạn vắng mặt, “chuyển động cao” là khung polyp có mức thay đổi đặc trưng ảnh lớn hơn 0.117 (ngưỡng tercile cố định từ tập phát triển).

Bảng 4: Kết quả trên C6 (trung bình theo chuỗi).

Phương pháp	Dice	Dice khung polyp	FP khung rỗng	Mất	Tái xuất hiện	Chuyển động cao
Native MedSAM2 (bank 7 khung)	0.617	0.599	0.486	0.283	0.462	0.563
EMA (1 khe, gain hằng)	0.603	0.623	0.587	0.268	0.525	0.588
Kalman (1 khe)	0.602	0.613	0.555	0.284	0.483	0.579

Bảng 5: So sánh cặp theo chuỗi trên C6 ($n = 8$, Wilcoxon; trong ngoặc: số chuỗi cao hơn / thấp hơn).

So sánh	Dice	Dice khung polyp	FP khung rỗng	Tái xuất hiện
Kalman – native	−0.014 (7/1), $p = 0.20$	+0.015 (6/2), $p = 0.31$	+0.069 (2/1), $p = 0.50$	+0.021 (4/3), $p = 0.58$
Kalman – EMA	−0.001 (7/1), $p = 0.20$	−0.010 (5/3), $p = 0.74$	−0.032 (0/3), $p = 0.25$	−0.042 (3/4), $p = 0.22$

Với $n = 8$, không khác biệt nào có ý nghĩa thống kê. Bộ nhớ một khe (Kalman và EMA) cho Dice khung polyp và Dice ở khung chuyển động cao

nhỉnh hơn ngân hàng bảy khung, nhưng làm tăng dương tính giả trên khung rỗng, nên Dice tổng thể thấp hơn native một chút. Kalman cao hơn native ở 7/8 chuỗi; trung bình thấp hơn là do một chuỗi (seq21). Bảng 3 cho thấy lợi ích lớn nhất ở seq18 (Dice khung polyp 0.647 so với 0.560) và seq22 (0.746 so với 0.710). Hai chuỗi seq20 và seq23 thất bại với cả ba phương pháp (Dice khung polyp không quá 0.1): đây là lỗi theo dõi mà không bộ nhớ nào khắc phục được. Về tích lũy lỗi, tính gộp trên các khung polyp: native mất 0.247 số khung, đoạn mất trung bình 7.1 khung, 0.64 số đoạn được khôi phục; Kalman lần lượt 0.244, 7.7 và 0.60, tức không khác biệt trên các video ngắn của C6.

Phân tích lỗi trên seq21. Đây là chuỗi mà Kalman thấp hơn native nhiều nhất (0.448 so với 0.685). Trên các khung có polyp, Kalman ngang native (Dice khung polyp 0.751 so với 0.757), tức không có sụp đổ theo dõi. Thiệt hại nằm ở đoạn rỗng dài ở cuối chuỗi (23 khung rỗng trong 47): object score của native chuyển âm nên native dự đoán rỗng, còn bộ nhớ một khe giữ object score dương và tiếp tục xuất mặt nạ. Trên C6, object score gọi 38% (native), 62% (EMA) và 57% (Kalman) số khung rỗng là có đối tượng: dương tính giả là bài toán hiện diện, không phải bài toán theo dõi.

5.1.3 Gain Kalman so với gain hằng

Theo phân tích trạng thái dừng, gain của bộ lọc chưa huấn luyện hội tụ về hai giá trị gần như cố định (khoảng 0.92 khi có đối tượng, 0.27 khi vắng mặt). Do trên tập phát triển, gain trung bình là 0.835 (độ lệch chuẩn 0.144) và 0.266. Trên C6, Kalman và EMA dùng gain này gần như trùng nhau: Dice -0.001 ($p = 0.20$), Dice khung polyp -0.010 ($p = 0.74$), EMA tốt hơn ở tái xuất hiện (-0.042) và Kalman có ít dương tính giả hơn (-0.032), đều không có ý nghĩa. Như đã nêu ở mục 3.2.2, sự hòa nhau này được kỳ vọng theo cách xây dựng: hai phương pháp chỉ khác ở gain chuyển tiếp, xác suất hiện diện mềm so với ngưỡng cứng, và chế độ `-skip_absent`. Kết luận rút ra là lợi ích (nếu có) đến từ *trạng thái lặp một khe*, không phải từ dạng Kalman.

5.1.4 Bộ nhớ và tốc độ

MedSAM2 gốc giữ bộ nhớ của mọi khung (khoảng 0.13 MiB mỗi khung) nhưng chỉ đọc tối đa bảy bộ nhớ (0.875 MiB ở bf16). Bộ nhớ một khe giữ anchor, mã vị trí, trung bình và phương sai ở float32 (0.754 MiB), cố định theo độ dài video, và chỉ đưa hai bản đồ bộ nhớ vào memory attention thay vì bảy, nên chạy nhanh hơn khoảng 10%.

Bảng 6: Bộ nhớ giữ lại cuối video và tốc độ lan truyền trên C6 (Apple MPS, đo bằng `infer.py`).

Phương pháp	Bộ nhớ giữ lại (MiB)	Khung/giây
Native MedSAM2	4.5–8.1 (tăng theo độ dài video)	20.2
EMA (1 khe)	0.754 (cố định)	22.3
Kalman (1 khe)	0.754 (cố định)	22.1

5.1.5 Đánh giá hai vùng trên REFUGE

Dữ liệu VA dự kiến theo Cai và cộng sự [3] dùng mặt nạ ba lớp mã hóa 0/128/255 và tỷ số $A/N = VA/(VA + \text{đường thở})$. REFUGE (ảnh đáy mắt) dùng cùng mã hóa với hai vùng kề nhau (vành đĩa thị và cup), nên tỷ số $\text{cup}/(\text{cup} + \text{vành})$ có cùng dạng với A/N . Mặt nạ được chuyển bằng ánh xạ 255:0 128:1 0:2. Vì vành là phần đĩa thị không thuộc cup, MedSAM2 gốc được prompt bằng hộp bao của đĩa thị và của cup (`-nested_labels`), rồi vành = đĩa – cup; cấu trúc này khớp với A/N , nơi mũi hòng = $VA + \text{đường thở}$.

Bảng 7: Hai vùng trên REFUGE val (400 ảnh, MedSAM2 gốc, prompt lồng nhau).

Dice cup	IoU cup	Dice vành	IoU vành	MAE tỷ số	RMSE tỷ số	Pearson	Spearman
0.917	0.850	0.924	0.861	0.019	0.024	0.98	0.98

Bảng 8: Sai số tỷ số theo chất lượng phân đoạn (Dice nhỏ hơn của hai vùng).

Dice nhỏ nhất	Số ảnh	MAE tỷ số
< 0.8	12	0.042
0.8–0.9	139	0.023
≥ 0.9	249	0.015

Tỷ số thật nằm trong khoảng 0.05–0.73 (trung bình 0.23). Sai số có dấu trung bình chỉ +0.008 (không lệch hệ thống), sai số tuyệt đối trung vị 0.016, 90% số ảnh có sai số dưới 0.039 và lớn nhất là 0.074. Sai số tăng khi phân đoạn kém đi, nhưng ngay cả khi Dice tốt (≥ 0.9) vẫn còn khoảng 1.5 điểm phần trăm. Như vậy trên dữ liệu thay thế, phân đoạn tốt cho phép đo tỷ số tốt; REFUGE chưa cho thấy sự lệch giữa Dice và phép đo, và chưa có cơ sở cho một hàm mất mát nhận biết phép đo. Câu hỏi này cần được kiểm tra lại trên dữ liệu VA, nơi ranh giới giữa VA và đường thở có thể khó hơn.

5.2 Kỹ năng và kiến thức thu thập được

Về kiến thức: kiến trúc bộ nhớ của SAM2/MedSAM2, lý thuyết bộ lọc Kalman và quan hệ của nó với trung bình trượt mũ, các hướng cải tiến bộ nhớ trong phân đoạn video và bài toán đánh giá phi đại VA. Về lập trình: mở rộng một mã nguồn nghiên cứu lớn bằng mixin, xử lý dữ liệu video y tế, tìm và sửa lỗi khó thấy (sai thứ tự khung, lỗi đồng bộ trên MPS, cách prompt sai trên REFUGE) và viết kiểm thử cho đường dữ liệu. Về phương pháp: tách tập phát triển và tập đánh giá theo trung tâm, dùng baseline đối chứng (EMA) để xác định đúng nguồn gốc cải thiện, dừng lại mọi bảng từ dữ liệu đã lưu, kiểm tra kết quả không phụ thuộc máy chạy, và diễn giải kết quả không vượt quá điều kiện số liệu cho phép.

5.3 Hướng phát triển tiếp theo

1. **Dữ liệu VA dạng video** cùng giao thức tỷ số và ngưỡng phân độ; đây sẽ là tập kiểm tra sạch đầu tiên.
2. **Quyết định ghi bộ nhớ học được**: học khi nào nên ghi một khung vào trạng thái (từ object score, IoU dự đoán, mức thay đổi mặt nạ), với mục tiêu định trước trên tập phát triển; huấn luyện bộ cập nhật trên video thật bằng `train_kalman.py`.
3. **Baseline mạnh hơn**, trước hết là EMA-SAM, rồi bank bảy khung đã cắt tĩa, SAMURAI, SAM2Long, DAM4SAM; prompt thực tế (hộp nhiều, điểm, bộ phát hiện).
4. **Hàm mất mát nhận biết phép đo** chỉ khi dữ liệu VA cho thấy Dice và sai số tỷ số lệch nhau.

Tài liệu tham khảo

- [1] MedSAM2: Segment Anything in 3D Medical Images and Videos. <https://arxiv.org/abs/2504.03600>.
- [2] SAM 2: Segment Anything in Images and Videos. <https://arxiv.org/abs/2408.00714>.
- [3] Cai và cộng sự (2024). Deep Learning-Based Quantification of Adenoid Hypertrophy and Its Correlation with Apnea-Hypopnea Index in Pediatric Obstructive Sleep Apnea. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11687100/>.
- [4] PolypGen: A multi-center polyp detection and segmentation dataset. <https://arxiv.org/abs/2106.04463>.
- [5] REFUGE: Retinal Fundus Glaucoma Challenge, phân đoạn đĩa thị và cup trên ảnh đáy mắt.
- [6] SAM2Long: Enhancing SAM 2 for Long Video Segmentation. ICCV 2025.
- [7] DAM4SAM: A Distractor-Aware Memory for Visual Object Tracking with SAM2. <https://arxiv.org/abs/2509.13864>.
- [8] EMA-SAM. <https://arxiv.org/abs/2510.18213>.
- [9] SAMURAI: Adapting Segment Anything Model for Zero-Shot Visual Tracking with Motion-Aware Memory. <https://arxiv.org/abs/2411.11922>.
- [10] Recurrent Dynamic Embedding for Video Object Segmentation (RDEVOS). <https://arxiv.org/abs/2205.03761>.
- [11] LiVOS: Light Video Object Segmentation with Gated Linear Matching. <https://arxiv.org/abs/2411.02818>.

- [12] R. E. Kalman (1960). A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering.
- [13] Recurrent Kalman Networks: Factorized Inference in High-Dimensional Deep Feature Spaces. <https://arxiv.org/abs/1905.07357>.
- [14] KalmanNet: Neural Network Aided Kalman Filtering for Partially Known Dynamics. <https://arxiv.org/abs/2107.10043>.